Требование 1

```sql
CREATE OR ALTER TRIGGER trg_Rentals_AutoStatusLog
ON dbo.Rentals
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.RentalStatusLog (RentalId, OldStatus, NewStatus, ChangedAt, ChangedBy)
    SELECT
        i.RentalId,
        d.Status AS OldStatus,
        i.Status AS NewStatus,
        GETDATE(),
        SYSTEM_USER
    FROM inserted i
    INNER JOIN deleted d ON i.RentalId = d.RentalId
    WHERE i.Status <> d.Status
       OR (i.Status IS NOT NULL AND d.Status IS NULL);
END;
GO
```

Требование 2

```sql
CREATE OR ALTER FUNCTION dbo.fn_CalculateEstimatedRange
(
    @MaxRangeKm INT,
    @BatteryLevel INT
)
RETURNS INT
AS
BEGIN
    RETURN CAST(@MaxRangeKm * (@BatteryLevel / 100.0) AS INT);
END;
GO
```

Требование 3

```sql
CREATE OR ALTER PROCEDURE dbo.usp_CreateRental
    @CustomerId INT,
    @CarId INT,
    @DriverId INT,
    @StartOdometer DECIMAL(10,2),
    @BaseCost DECIMAL(10,2)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRAN;

        DECLARE @CarStatus NVARCHAR(20), @BatteryLevel INT;
        SELECT @CarStatus = Status, @BatteryLevel = BatteryLevel
        FROM dbo.Cars WHERE CarId = @CarId;

        DECLARE @ErrorMessage NVARCHAR(255);

        IF @CarStatus <> 'Available'
        BEGIN
            SET @ErrorMessage = FORMATMESSAGE('Автомобиль недоступен для аренды. Текущий статус: %s', @CarStatus);
            THROW 50001, @ErrorMessage, 1;
        END

        DECLARE @LicenseExpiry DATE;
        SELECT @LicenseExpiry = LicenseExpiry FROM dbo.Drivers WHERE DriverId = @DriverId;

        IF @LicenseExpiry < CAST(GETDATE() AS DATE)
            THROW 50002, 'Срок действия водительского удостоверения истёк.', 1;

        IF @BatteryLevel < 30
            THROW 50003, 'Недостаточный уровень заряда батареи (минимум 30%) для начала аренды.', 1;

        INSERT INTO dbo.Rentals (CustomerId, CarId, DriverId, StartAt, StartOdometer, BaseCost, Status, CreatedAt)
        VALUES (@CustomerId, @CarId, @DriverId, GETDATE(), @StartOdometer, @BaseCost, 'Active', GETDATE());

        DECLARE @NewRentalId INT = SCOPE_IDENTITY();

        UPDATE dbo.Cars SET Status = 'Rented' WHERE CarId = @CarId;

        COMMIT TRAN;
        SELECT @NewRentalId AS RentalId, 'Аренда успешно оформлена' AS Message;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRAN;
        THROW;
    END CATCH
END;
GO
```

Требование 4

```sql
CREATE OR ALTER PROCEDURE dbo.usp_CompleteRental
    @RentalId INT,
    @EndOdometer DECIMAL(10,2),
    @ExtraCost DECIMAL(10,2) = 0,
    @IsAccident BIT = 0
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRAN;

        DECLARE @CurrentStatus NVARCHAR(20), @CarId INT, @DriverId INT, @StartOdometer DECIMAL(10,2), @BaseCost DECIMAL(10,2);

        SELECT @CurrentStatus = Status, @CarId = CarId, @DriverId = DriverId, @StartOdometer = StartOdometer, @BaseCost = BaseCost
        FROM dbo.Rentals WHERE RentalId = @RentalId;

        IF @CurrentStatus <> 'Active'
            THROW 50004, 'Нельзя завершить аренду, которая не находится в статусе Active.', 1;

        DECLARE @DistanceCovered DECIMAL(10,2) = @EndOdometer - @StartOdometer;
        DECLARE @TotalCost DECIMAL(10,2) = @BaseCost + @ExtraCost;

        UPDATE dbo.Rentals
        SET EndAt = GETDATE(),
            EndOdometer = @EndOdometer,
            DistanceCovered = @DistanceCovered,
            ExtraCost = @ExtraCost,
            TotalCost = @TotalCost,
            Status = 'Completed'
        WHERE RentalId = @RentalId;

        UPDATE dbo.Cars
        SET CurrentMileage = CurrentMileage + @DistanceCovered
        WHERE CarId = @CarId;

        UPDATE dbo.Drivers
        SET TotalRents = TotalRents + 1,
            TotalDistance = TotalDistance + @DistanceCovered,
            AccidentCount = AccidentCount + CASE WHEN @IsAccident = 1 THEN 1 ELSE 0 END
        WHERE DriverId = @DriverId;

        COMMIT TRAN;
        SELECT 'Аренда успешно завершена, статистика обновлена' AS Message;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRAN;
        THROW;
    END CATCH
END;
GO
```

Требование 5

```sql
CREATE OR ALTER TRIGGER trg_Cars_AutoChargeOnLowBattery
ON dbo.Cars
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE c
    SET Status = 'Charging'
    FROM dbo.Cars c
    INNER JOIN inserted i ON c.CarId = i.CarId
    INNER JOIN deleted d ON c.CarId = d.CarId
    WHERE i.BatteryLevel < 20
      AND d.BatteryLevel >= 20
      AND i.Status <> 'Charging';
END;
GO
```

Требование 6

```sql
CREATE OR ALTER FUNCTION dbo.fn_GetDriverReliabilityRating
(
    @DriverId INT
)
RETURNS INT
AS
BEGIN
    DECLARE @TotalRents INT, @AccidentCount INT, @Rating INT;

    SELECT @TotalRents = TotalRents, @AccidentCount = AccidentCount
    FROM dbo.Drivers WHERE DriverId = @DriverId;

    IF @TotalRents = 0 OR @TotalRents IS NULL
        RETURN 0;

    SET @Rating = CAST(((@TotalRents - @AccidentCount) * 100.0 / @TotalRents) AS INT);

    RETURN CASE WHEN @Rating < 0 THEN 0 ELSE @Rating END;
END;
GO
```

Требование 7

```sql
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRAN;

        DECLARE @CarId INT, @ChargeFrom INT;

        SELECT @CarId = CarId, @ChargeFrom = ChargeFrom
        FROM dbo.ChargingLogs WHERE LogId = @LogId;

        IF @ChargeTo < @ChargeFrom
            THROW 50005, 'Уровень заряда после зарядки (ChargeTo) не может быть меньше начального (ChargeFrom).', 1;

        DECLARE @EnergyConsumed DECIMAL(6,2) = CAST((@ChargeTo - @ChargeFrom) AS DECIMAL(6,2));

        UPDATE dbo.ChargingLogs
        SET ChargingEnd = GETDATE(),
            ChargeTo = @ChargeTo,
            EnergyConsumedKwh = @EnergyConsumed
        WHERE LogId = @LogId;

        UPDATE dbo.Cars
        SET BatteryLevel = @ChargeTo,
            Status = 'Available'
        WHERE CarId = @CarId;

        COMMIT TRAN;
        SELECT 'Зарядка успешно завершена, автомобиль доступен' AS Message;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRAN;
        THROW;
    END CATCH
END;
GO
```

Требование 8

```sql
CREATE OR ALTER TRIGGER trg_Rentals_PreventInvalidCarRental
ON dbo.Rentals
INSTEAD OF INSERT
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (
        SELECT 1
        FROM inserted i
        INNER JOIN dbo.Cars c ON i.CarId = c.CarId
        WHERE c.Status IN ('Maintenance', 'Accident', 'Charging')
    )
    BEGIN
        THROW 50006, 'Ошибка безопасности: Невозможно оформить аренду. Автомобиль находится в статусе обслуживания, аварии или зарядки.', 1;
    END

    INSERT INTO dbo.Rentals (CustomerId, CarId, DriverId, StartAt, EndAt, StartOdometer, EndOdometer, DistanceCovered, BaseCost, ExtraCost, TotalCost, Status, CreatedAt)
    SELECT CustomerId, CarId, DriverId, StartAt, EndAt, StartOdometer, EndOdometer, DistanceCovered, BaseCost, ExtraCost, TotalCost, Status, CreatedAt
    FROM inserted;
END;
GO
```

№	Выбранный объект	Почему именно это	Почему не другое

1	**Триггер**	Журнал должен заполняться сам, когда меняется статус аренды. Оператор может забыть или ошибиться, а триггер сработает всегда автоматически	Процедура не сработает, если кто-то изменит статус напрямую через SQL. Функция не может записывать данные в таблицу
2	**Функция**	Нужна формула, которую можно использовать в любом запросе, как калькулятор	Триггер и процедура не могут возвращать значение для использования в SELECT
3	**Процедура**	Это сложный процесс с множеством проверок: статус авто, срок прав, заряд батареи. Нужно всё сделать за один раз и либо полностью, либо никак	Триггер не принимает параметры от пользователя. Функция не может изменять данные в таблицах
4	**Процедура**	Завершение аренды — это много действий: обновить аренду, пробег авто, статистику водителя. Всё должно выполниться вместе, иначе данные будут неверными	Триггер срабатывает сам, а здесь нужно действие оператора. Функция не может делать несколько обновлений
5	**Триггер**	Когда заряд падает ниже 20%, авто должен сам перейти в статус «Заряжается». Никто не должен об этом помнить -система сама следит	Процедура потребовала бы добавлять проверку во все места, где меняется заряд
6	**Функция**	Рейтинг — это просто расчёт по формуле. Нужен в отчётах, его можно вызвать в любом запросе к водителям	Триггер и процедура слишком сложные для простого расчёта. Они не возвращают значение для SELECT
7	**Процедура**	Завершение зарядки требует проверки и обновления двух таблиц: журнал зарядки и батарея авто. Всё должно быть атомарно	Триггер не принимает параметры от оператора. Функция не может обновлять данные
8	**Триггер**	Защита должна работать на уровне базы данных, чтобы никто не мог арендовать неисправное авто - ни через программу, ни напрямую через SQL	Процедура защитит только если все идут через неё. Прямой INSERT обойдёт защиту. Функция не может блокировать вставку